MediCare RAG Assistant

Low-Level Design Document

Implementation Blueprint — Module Level Design

FastAPI + LangGraph + ChromaDB + GPT-4o

Version 1.0 | Internship Project

1. Module-Level Design

The system is decomposed into 8 independent modules, each with a single responsibility. Modules communicate through Pydantic schemas ensuring type safety at every boundary.

1.1 Document Processing Module

File: app/services/ingestion.py — Class: DocumentIngestionService

- Accepts a local PDF file path as input
- Delegates extraction to PyMuPDF (fitz.open)
- Returns a list of (page_text, page_number) tuples
- Handles empty pages gracefully (skipped, not errored)

1.2 Chunking Module

Embedded within DocumentIngestionService.chunk_document()

- Uses LangChain RecursiveCharacterTextSplitter
- Processes one page at a time to preserve page metadata
- Returns List[dict] with keys: id (UUID), text, metadata {source, page, chunk_index}

Chunking Parameters (configurable via .env)

```
CHUNK_SIZE=800       # Characters per chunk
CHUNK_OVERLAP=150    # Overlap between consecutive chunks
Separators: ['\n\n', '\n', '. ', ' ', '']
```

1.3 Embedding Module

Uses OpenAI text-embedding-3-small via LangChain OpenAIEmbeddings wrapper.

- embed_documents(texts) for batch indexing (returns List[List[float]])
- embed_query(text) for single query embedding
- Both methods called via the same embeddings instance for consistency

1.4 Vector Storage Module

ChromaDB PersistentClient with cosine metric. Key operations:

- collection.add(ids, embeddings, documents, metadatas) — batch 50 chunks
- collection.query(query_embeddings, n_results, include=[...]) — retrieval
- collection.count() — readiness check

- chroma_client.delete_collection() — reset operation

1.5 Retrieval Module

File: app/services/retrieval.py — Class: RetrievalService

- Embeds query string → calls collection.query → converts cosine distance to score
- score = max(0.0, 1.0 − cosine_distance) → range [0, 1]
- Returns (List[SourceDocument], best_confidence_score)

1.6 Query Processing Module

File: app/services/graph.py — Functions: input_node, generate_node

- input_node: System prompt + query → GPT-4o (temp=0) → intent label string
- generate_node: Formats context string from top-K docs → builds message history → GPT-4o (temp=0.3)
- Conversation history: last 6 messages (3 turns) appended before current query

1.7 Graph Execution Module

File: app/services/graph.py — Function: build_rag_graph()

- StateGraph(GraphState) — creates a typed directed graph
- Nodes added: input_node, retrieve_node, generate_node, escalate_node
- Entry point: input_node
- Conditional edge: retrieve_node → {generate, escalate}
- Terminal edges: generate_node → END, escalate_node → END
- graph.compile() returns an executable Runnable

1.8 HITL Module

File: app/services/hitl.py — Class: HITLService

- In-memory dict _ticket_store: Dict[str, HITLTicket]
- create_ticket() — generates TKT-XXXXXXXX ID, stores ticket
- resolve_ticket() — sets status=resolved, human_response, resolved_at
- should_escalate() — returns (bool, EscalationReason | None)

2. Data Structures

2.1 Document & Chunk Representation

Chunk dict (internal):

```
{ "id": "uuid4-string",
  "text": "...policy text...",
  "metadata": { "source": "healthcare_policy", "page": 3, "chunk_index": 1 } }
```

SourceDocument (Pydantic — returned to client):

```
class SourceDocument(BaseModel):
    content: str
    page: Optional[int]
    source: Optional[str]
    score: Optional[float]   # cosine similarity 0..1
```

2.2 Query-Response Schema

QueryRequest:

```
session_id: str          # UUID per browser session
message: str             # User input
conversation_history: List[ChatMessage]   # Last N turns
```

QueryResponse:

```
session_id, answer, intent, sources, confidence,
escalated, escalation_reason, hitl_ticket_id, processing_time_ms
```

2.3 GraphState TypedDict

GraphState (flows between LangGraph nodes):

```
session_id: str
query: str
conversation_history: List[ChatMessage]
intent: Optional[str]          # set by input_node
retrieved_docs: List[SourceDocument]  # set by retrieve_node
confidence: float              # set by retrieve_node
answer: str                    # set by generate/escalate_node
escalated: bool
escalation_reason: Optional[str]
hitl_ticket_id: Optional[str]
```

```
processing_start: float        # timestamp for latency calc
```

2.4 HITLTicket Schema

HITLTicket (Pydantic):

```
ticket_id: str              # TKT-XXXXXXXX format
session_id: str
original_query: str
escalation_reason: EscalationReason
context_snippets: List[str]  # Top-3 retrieved chunks
status: str                 # open | in_progress | resolved
created_at: datetime
human_response: Optional[str]
resolved_at: Optional[datetime]
```

3. LangGraph Workflow Design

3.1 Node Specifications

Node	Input (from State)	Output (to State)
input_node	query, conversation_history	intent (string)
retrieve_node	query	retrieved_docs, confidence
generate_node	query, retrieved_docs, conversation_history	answer, escalated=False
escalate_node	query, retrieved_docs, session_id, escalation_reason	answer, escalated=True, hitl_ticket_id

3.2 Edge Definitions

Linear edges (unconditional):

- START → input_node (set_entry_point)
- input_node → retrieve_node
- generate_node → END
- escalate_node → END

Conditional edge:

- retrieve_node → should_escalate_router() → 'generate' | 'escalate'

3.3 State Object Flow

State evolution through nodes:

```
START:   { session_id, query, conversation_history }
After input_node:   + intent
After retrieve_node: + retrieved_docs, confidence
After generate_node: + answer, escalated=False
  OR
After escalate_node: + answer, escalated=True, hitl_ticket_id, escalation_reason
```

4. Conditional Routing Logic

4.1 Router Function: should_escalate_router()

Evaluated after retrieve_node, in the following priority order:

Priority	Condition	Reason Enum
1 (highest)	Intent == emergency OR sensitive keyword in query	SENSITIVE_KEYWORDS
2	confidence < CONFIDENCE_THRESHOLD (default: 0.65)	LOW_CONFIDENCE
3	len(retrieved_docs) == 0	MISSING_CONTEXT
4 (default)	All checks pass	→ generate_node (no escalation)

4.2 Sensitive Keywords (Configurable)

- Default list: lawsuit, sue, legal, malpractice, critical, emergency, death, dying
- Configurable via HITL_KEYWORDS environment variable (comma-separated)
- Case-insensitive substring match

4.3 Answer Generation Criteria

A query proceeds to generate_node when ALL of the following are satisfied:

- No sensitive keywords detected in the query
- At least one chunk was retrieved (retrieved_docs is non-empty)
- Best cosine similarity score >= CONFIDENCE_THRESHOLD (0.65)
- Intent is not 'emergency'

5. HITL Design

5.1 When Escalation is Triggered

Escalation is determined in the should_escalate_router after retrieval completes. The three trigger conditions are evaluated in priority order as described in Section 4.

5.2 What Happens After Escalation

- escalate_node creates a HITLTicket with all relevant context
- A standardized escalation message is returned to the user with ticket ID
- The ticket appears in the HITL Dashboard (React /tickets view)
- Human agent can expand ticket, view context snippets, and type a resolution
- POST /api/v1/hitl/resolve marks ticket resolved and stores human response

5.3 Human Response Integration

Currently, the human response is stored in the ticket and visible to users who re-check their ticket. In a production system, this would:

- Trigger a webhook or email notification to the customer
- Log the resolution to a CRM (Salesforce/HubSpot)
- Feed the query-response pair back into a fine-tuning dataset

6. API & Interface Design

6.1 REST API Endpoints

Endpoint	Method + Path	Description
Status Check	GET /api/v1/status	Returns vector store readiness, chunk count, model info
Chat	POST /api/v1/chat	Main RAG pipeline — accepts QueryRequest, returns QueryResponse
Ingest PDF	POST /api/v1/ingest	Multipart file upload → ingestion pipeline
Reset KB	DELETE /api/v1/ingest/reset	Wipes ChromaDB collection
List Tickets	GET /api/v1/hitl/tickets	Optional ?status=open|resolved filter
Get Ticket	GET /api/v1/hitl/tickets/{id}	Single ticket detail
Resolve	POST /api/v1/hitl/resolve	Marks ticket resolved with human response

6.2 React Frontend Pages

Page	Description
ChatPage	Message thread UI with auto-scroll. Shows intent badges, confidence bars, source panels, HITL ticket IDs.
IngestPage	Drag-and-drop PDF uploader with progress bar. Shows ingestion result (chunks stored, document name).
TicketsPage	Filterable ticket list. Expandable cards with context snippets and agent response form.
StatusPage	System health dashboard. Shows 4 stat cards and a component health table.

7. Error Handling

Scenario	Where Handled	Response
Non-PDF file uploaded	FastAPI routes.py	HTTP 400: 'Only PDF files are supported'
ChromaDB empty on chat	FastAPI routes.py	HTTP 503: 'Knowledge base not ready'
OpenAI API failure	graph.py try/except	Fallback intent=general_info; error surfaced to UI
Zero chunks retrieved	Routing logic	Escalated to HITL with MISSING_CONTEXT reason
Intent classification fails	input_node try/except	Defaults to general_info, pipeline continues
Empty message sent	FastAPI validator	HTTP 422: 'Message cannot be empty'
Ticket not found	HITL routes	HTTP 404: 'Ticket not found'
LLM generation error	generate_node	Exception propagated → HTTP 500 with detail